

DATA INTEGRATION PLAYBOOK

How to turn the OnyxTaxSources collection into a working system · August 2026

What this is: you've collected ~49 files across 8 folders. This document says what each one is *for*, how to feed them to an AI without getting garbage, and what order to build in. **Read §1 and §2 before writing any code** — they contain the one architectural decision that determines whether this product works or gets you sued.

1. THE ONE RULE THAT MATTERS

The AI never computes tax. The engine computes tax. The AI explains what the engine computed.

Everything you collected splits cleanly along that line, and mixing the two sides is the single biggest failure mode in this category:

	DETERMINISTIC SIDE (code)	LANGUAGE SIDE (AI)
Job	Produce numbers	Produce sentences
Fed by	tax_engine_params.json, the formulas reference, T4127	The CRA guides, folios, and rate pages — as a citation corpus
Must be	Exact, testable, reproducible, auditable	Clear, hedged, sourced
Failure mode	Wrong refund → CRA reassessment	Confidently wrong advice → reputational + legal
Test by	Persona suite with locked expected outputs	Human review + mandatory citations

Concretely: when a user asks "should I put \$5,000 in my RRSP?", the AI does not calculate the answer. The engine runs `tax(income) - tax(income - 5000)` and returns \$1,482.50. The AI then writes: "Contributing \$5,000 would reduce your tax by about \$1,482 because it drops your last dollars out of the 29.65% bracket." The number came from code. Only the sentence came from the model.

2. WHAT YOU ACTUALLY HAVE — INVENTORY BY PURPOSE

Folder	Role in the system	Feed to AI?
00-engine tax_engine_params.json	Runtime config. The engine imports this at boot. All brackets, credits, CPP/EI, RRSP limits, 12 provinces × 2 years.	YES — small, structured, exact. This is the single best file you own.
00-engine Formulas reference PDF	Implementation spec. Tells the developer/AI what code to write: the pipeline, bracket function, credit math, simulator formulas.	YES — this is the build brief.

01-legislation Act, Regulations, budget annex	Ultimate authority. Consulted by humans when a rule is genuinely disputed.	NO — 5,000+ pages, will blow any context window and return noise.
02-cra-guidance T4002, RC4022, folio index, CCA	Copilot corpus. Plain-language explanations of rules — exactly what a Q&A feature should quote from.	YES, chunked — this is your RAG source (see §4).
03-rates-amounts brackets, CPP, EI, RRSP/TFSA, interest, auto, meal, GST	Proof of the config. These are the primary sources the JSON was verified against — your audit trail.	NO for computing (use the JSON instead). Keep as evidence.
04-filing-specs T1, ON428, worksheets, T4127, XML specs	Output contract + answer key. T4127 is CRA's own math; the forms are what your output must reproduce line-for-line.	YES, selectively — feed T4127 when building withholding logic, forms when mapping outputs.
05-quebec	Quarantined future scope. Do not let this leak into the main engine path.	NO — not until Quebec is a decision.
06-apis-data FX CSV, Valet OpenAPI, Flinks	Live-data integration. The <code>.yaml</code> generates an API client; the CSV is real FX conversion data.	YES — the OpenAPI spec especially; AI generates clients from it well.
07-monitoring	Process, not data. The weekly/annual check schedule.	NO — it's a to-do list for a human.

3. BUILD ORDER — WHAT TO DO WITH IT, IN SEQUENCE

Step 1 — Stand up the calculator core (uses: JSON + formulas PDF)

Hand your AI tool the JSON and the formulas reference and ask for exactly this, nothing more:

```
Build a pure function: calculateTax(taxableIncome, province, year) -> { federalTax, provincialTax, total }
```

Requirements:

- Read ALL numbers from tax_engine_params.json. Hardcode nothing.
- Implement applyBrackets() per section 1 of the formulas reference.
- Use integer cents or a decimal type. Never floating point for money.
- Throw a clear error if the year or province is missing from config.
- No network calls, no AI calls, no side effects. Pure function.

This is deliberately tiny. It is also the foundation everything else sits on — if this is wrong, every feature above it is wrong.

Step 2 — Lock it with a persona suite (uses: T1 forms, T4127)

Before adding a single feature, build ~30 test cases with expected outputs:

```
single T4, $55k, ON      | two T4s, $40k + $25k, BC
T4 + $6k RRSP, $95k, AB | self-employed $80k, SK
investor: $50k + $12k eligible dividends + $8k capital gain, ON
senior: $70k with OAS clawback exposure, NS
each province at $60k, $120k, $250k
```

How to get trustworthy expected values: run the same scenarios through an existing certified product (Wealthsimple Tax is free) and record its output. Any disagreement is a bug in your engine until proven otherwise. This is the highest-leverage hour in the whole project — and it's exactly what CRA's certification test returns will do to you later, so you may as well practise now.

Step 3 — Add the simulators (uses: formulas PDF §10)

These are your differentiator and they need no certification. Critical implementation note, straight from the reference:

```
// WRONG - breaks when the contribution crosses a bracket:
saving = contribution * marginalRate

// RIGHT - run the full pipeline twice:
saving = totalTax(taxableIncome) - totalTax(taxableIncome - contribution)
```

Build RRSP first, then FHSA (same math, different caps), then capital-gains timing.

Step 4 — Wire the Tax Health Score to the engine (uses: formulas PDF §11)

The score is your invention, so it has no external authority — which means its credibility rests entirely on its dollar figures coming from the engine you just tested. Every point lost must map to an action card whose "+\$X" was computed, never estimated. Kill every hardcoded number from the demo (82/100, \$2, 431, +\$420).

Step 5 — Only now, the copilot (uses: 02-cra-guidance corpus)

Built last and constrained hardest — see §4.

4. HOW TO BUILD THE COPILOT SO IT DOESN'T BURN YOU

The pattern is **retrieval over your corpus, never free generation**:

1. **Chunk the guidance folder** — split T4002, RC4022, and any folios you pull into ~500-word passages with their source and page recorded.
2. **Embed and index them** in a vector store.
3. **On a user question**: retrieve the top passages, then prompt the model with a hard instruction — *answer only from these passages; cite the source; if they don't cover it, say you don't know and offer to connect a human*.
4. **Route anything numeric to the engine**. If the question implies a calculation ("how much would I save..."), the app calls the calculator and hands the model the result to narrate. The model never does arithmetic.
5. **Log every exchange** — question, retrieved passages, engine outputs, final answer. That log is your defence if an answer is ever disputed.

Three hard rules for the copilot: (1) **Never let it invent a number**. If a figure isn't from the engine or a cited passage, it doesn't ship. (2) **Label AI answers as AI** and keep the "not tax advice" framing consistent with your ToS. (3) **Never send more personal data than the question needs** — no SIN, no full income history, into a model prompt. Send the minimum, and prefer Canadian-region endpoints (see the legal checklist).

5. FEEDING FILES TO AN AI — THE TIERS

Tier	Files	How
Always in context	<code>tax_engine_params.json</code> , formulas reference	Small enough to paste whole. Include both at the start of any engine-building session.
Task-scoped	T4127, T1/ON428 forms, T4 XML spec, Valet <code>.yaml</code>	Attach only while working the matching feature. Then drop them — stale context degrades output.
Chunked into a vector store	T4002, RC4022, folios	Never pasted whole; retrieved passage-by-passage at question time.
Human-only, never fed	Income Tax Act, Regulations, forms/publications indexes, monitoring notes	Look things up by hand. Feeding these produces confident nonsense.

5.1 A prompt template that works

CONTEXT: I'm building a Canadian T1 tax engine. Attached: - <code>tax_engine_params.json</code> (verified CRA parameters, 2025 and 2026) - <code>Tax_Engine_Formulas_and_Data_REFERENCE.pdf</code> (the spec)
TASK: [one specific function]
CONSTRAINTS: - Every tax number comes from the JSON. Zero hardcoded values. - Money in integer cents. - Any parameter with <code>"_verified": false</code> must raise a warning, not be used silently. - Write the unit tests alongside, using the persona cases in <code>/tests</code>. - If the spec is ambiguous, ask me — do not guess a tax rule.

That last line matters more than it looks. An AI guessing a tax rule is exactly how a wrong refund reaches a real filer.

6. THE VERIFICATION LOOP (RUN THIS FOREVER)

Trigger	Action
Any engine change	Rerun the persona suite. All expected values must still match.
Using a parameter flagged <code>_verified: false</code>	Stop. Source it from CRA first, update the JSON, flip the flag, note the source page.
Every November	New CRA numbers publish → re-capture folder 03 → add a new year block to the JSON → rerun the suite. Formulas rarely change; only the tables do.
January & July	New T4127 edition → re-download → check withholding logic.
Weekly (from 07-monitoring)	Gazette, LEGISinfo, Finance draft legislation — tag anything new as PROPOSED, never IN_FORCE, until Royal Assent.

Known gaps that will block provincial tax right now: the JSON does not yet contain **provincial basic personal amounts**, **provincial dividend tax credit rates**, or **Ontario surtax thresholds**. Provincial tax computes wrong without them. Source these three before anything provincial reaches a user's screen. Full list in §15.2 of the formulas reference.

7. THE FIRST WEEK, CONCRETELY

1. **Day 1:** Repo skeleton. Drop `tax_engine_params.json` into `/config`. Write the loader that reads it and refuses to boot if a required year is missing.
2. **Day 2:** `applyBrackets()` + `calculateTax()`, federal only. Unit tests from the formulas reference worked examples.
3. **Day 3:** Add provincial (Ontario first). Immediately hit the missing-BPA gap — go source it, update the JSON, flip the flag. *This teaches the whole verification discipline in one afternoon.*
4. **Day 4:** Build the 30-persona suite. Cross-check against a certified product. Fix what disagrees.
5. **Day 5:** RRSP simulator using the correct two-pass formula. This is your first real user-facing feature — and it needs no certification.

At the end of that week you have a tested calculator and a working simulator: enough to put the Tax Health Score in front of real Canadians while certification is still a year away. That's the launch path from the strategy doc, and this data collection is what makes it possible.

Companion to *Tax Engine Formulas & Data Reference* (the spec), `tax_engine_params.json` (the config), and the *Canadian Legal Checklist*. Engineering guidance only — not tax or legal advice. Nothing here substitutes for NETFILE certification.